188. Best Time to Buy and Sell Stock IV

Approach 1: Dynamic Programming

Intuition

Dynamic programming (DP) is a popular method among hard-level problems. Its basic idea is to store the previous result to reduce redundant calculations. However, it is hard for beginners to think of the DP method. Below, a step-by-step tutorial on how to think of dp is introduced. If you are already familiar with dp, you can jump to the algorithm part to check out the actual implementation.

Generally, there are two ways to come up with a DP solution. One way is to start with a brute-force approach and reduce unnecessary calculations. Another way is to treat the stored results as "states", and try to jump from the starting state to the ending state. For beginners, it is recommended to start with the brute force approach. So, how to brute force solve this problem?

Back to (part of) the question:

Say you have an array for which the i-th element is the price of a given stock on day i. Design an algorithm to find the maximum profit. You may complete at most k transactions.

Cool, looks like we need to arrange at most k transactions. A natural idea is to iterate all the possible combinations of k transactions, and then find the best combination. As for those with less than k transactions, they are similar and can be considered later. A transaction consists of two parts: buying and selling. Therefore, we need to find 2k points in the stock line, k points for buying, and k points for selling.

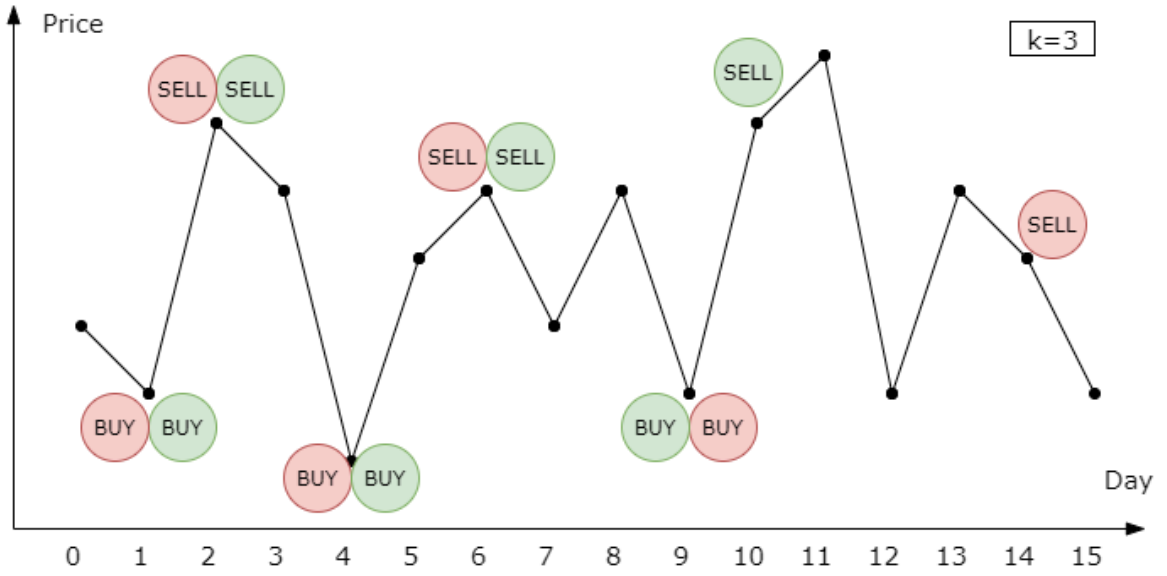
Now, we can roughly estimate the time complexity. Suppose there are n days in total, and we need to pick 2k days. The number of possible situations is about Cn2k=(2k)!(n−2k)!n!. It's not a good result because it involves factorial, which is likely to cause Time Limit Exceeded (TLE). Usually what we need is a polynomial one. However, it includes some invalid situations so the actual number is smaller.

Another problem is that what if `k * 2` is larger than n? In this case, we are not able to pick 2k points from n points, which means we will not reach the limit no matter how we try. Therefore, all we need to do is to iterate each day, and if the price of day `i` rises, buy the stock in i-1th day and sell it at ith day.

k⋅2≥n is a special case and can be addressed easily.

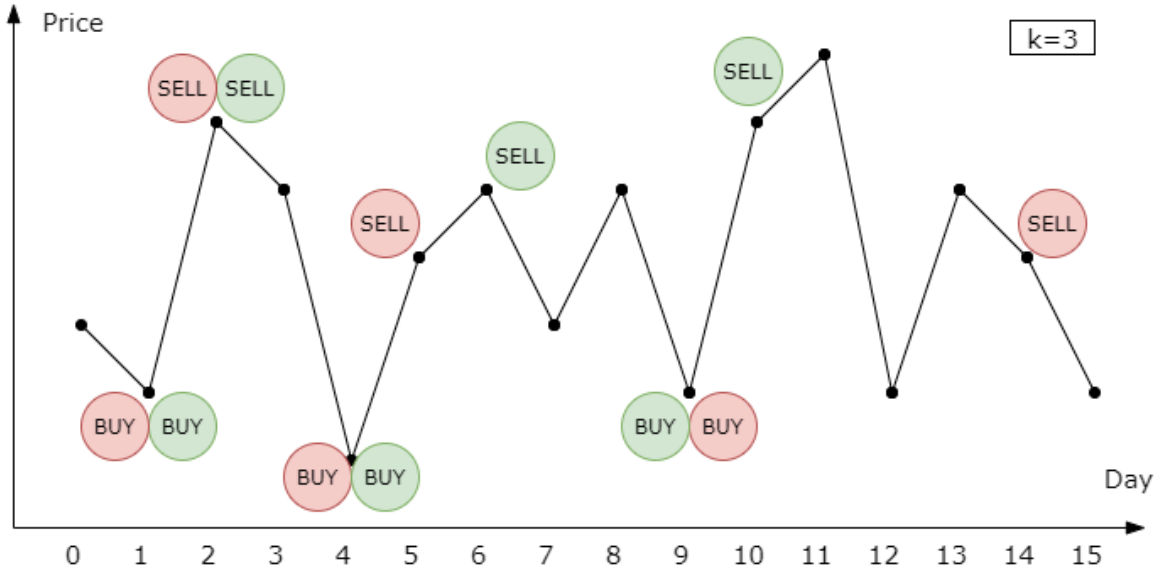
Back to our factorial number. The next step is to review our brute force approach and find out the possible redundant calculations. In our brute force approach, we need to iterate all the possible combinations and calculate the profit of each one to find the best. Can you find out where repeated calculations are?

Consider the following case, where the red color represents a possible combination, and the green represents another one:



The two combinations are the same before day 10. If we calculate the profits separately, we need to calculate the profit before day 10 twice. Here is where dp comes in! We can store the current balance on day 9, and reuse it later. Therefore, we can store the result in a hash map, where the key is the day number and the transactions we made before, and the value is the balance. Wait a minute, can we do better?

Consider another case:



The only difference is that the red sells stock at a lower price during the second transaction. Therefore, the red has a lower profit on day 10 than the green has. In this case, we need not calculate the rest profit of the red, since it can not beat the green in the future.

Therefore, we can compare those reds, and continue the next day with the one with the highest profit. However, we need to ensure that the best one will not be beaten by the "losers" in the future, so they should have the same "resources" at the time we store and compare the balances.

Hence, we can use three characteristics to store the profit: the day number, the transaction number used, and the stock holding status. You can use other representations of resources, such as using "the day remained" instead of "the day number". Feel free to try. Now, let's go to the algorithm part.

Algorithm

In the previous part, we introduced an intuitive idea from brute force to dp method, and here we need to decide the details of the algorithm.

We can either store the dp results in a dictionary or an array. An array costs less time to access and update than a dictionary, so we always prefer an array when possible.

Because of three needed characteristics (day number, transaction number used, stock holding status), a three-dimensional array is our choice. We can use `dp[day_number][used_transaction_number][stock_holding_status]` to represent our states, where `stock_holding_status` is a 0/1 number representing whether you hold the stock or not.

The value of `dp[i][j][l]` represents the best profit we can have at the end of the `i`-th day, with `j` remaining transactions to make and `l` stocks.

The next step is finding out the so-called "transition equation", which is a method that tells you how to jump from one state to another.

We start with `dp[0][0][0] = 0` and `dp[0][1][1]=-prices[0]`, and our final aim is max of `dp[n-1][j][0]` from `j=0` to `j=k`. Now, we need to fill out the entire array to find out the result. Assume we have gotten the results before day `i`, and we need to calculate the profit of day `i`. There are only four possible actions we can do on the day `i`: 1. keep holding the stock, 2. keep not holding the stock, 3. buy the stock, or 4. sell the stock. The profit is easy to calculate.

1. Keep holding the stock:

dp[i][j][1]=dp[i−1][j][1]

2. Keep not holding the stock:

dp[i][j][0]=dp[i−1][j][0]

3. Buying, when j>0:

dp[i][j][1]=dp[i−1][j−1][0]−prices[i]

4. Selling:

dp[i][j][0]=dp[i−1][j][1]+prices[i]

We can combine them together to find the maximum profit:

dp[i][j][1]=max(dp[i−1][j][1],dp[i−1][j−1][0]−prices[i])

dp[i][j][0]=max(dp[i−1][j][0],dp[i−1][j][1]+prices[i])

Awesome! Now we can use for-loop to calculate the whole dp array and achieve our final result. Remember to solve the special cases when k⋅2≥n.